COMPILER TRANSFORMATIONS

Compiler transformation also known as program transformation or program optimization refers to the process of modifying a program's code or structure to improve its performance, efficiency or other desirable properties.

✓ PARALLELISM:

It refers to the ability to execute multiple tasks or operations simultaneously, either on multiprocessor cores or in distirbuted computing environments.
It involves breaking down a program into smaller units of work that can be executed concurrently, thereby improving overall performance and reducing the execution time.

PARALLELIZATION TRANSFORMATION: Parallelization transformations aim to exploit the parallelism in a program to execute multiple tasks or operations simultaneosly.

Eg: Loop parallelization,
data parallelization [Vectorization]
Pipelining
Automatic parallelization.

AUTOMATIC PARALLELIZATION

Automatic parallelization is a compiler optimization technique that automatically identifies and transforms sequential code into parallel code. It encomposses various transformations including loop parallelization,

task parallelization and data parallelism, to exploit parallelism without manual intervention.

Before performing parallelisation we need to satisfy few synchronisation conditions, because one statement can affect another statement.

So for this we have to perform DATA DEPENDENCY ANALYSIS

DATA DEPENDENCY ANALYSIS

✓ TRANSFORMATIONS BEFORE DEPENDENCY ANALYSIS

→ Array subscripts should be linear functions of loop variables.

→ Loop lower bound should be one and the loop increment should be one.

→ A few loop transformations are carried out to ensure the above

- ★ Loop Normalization
- ★ Induction variable substitution.
- ★ Expression folding and foward substitution.

LOOP NORMALIZATION

Loop Normalization, also known as compiler's loop canonicalization is a transformation technique used in compiler optimization to convert a loop into a more structured and canonical forms. It aims to simplify subsequent analyses and optimizations by applying a set of standard loop transformations.

Lower bound → 1 Loop increment → 1

ORIGINAL LOOP:

```
for (int i = 0; i <= 100; i++)
{
  for (int j = 1; j <= 300; j += 3)
    {
      x[j] = x[j] + 5;
      y[j+4] = y[j] + 8;
    }
}
```

NORMALIZED LOOP:

```
for (int i = 1; i <= 100; i++) {
  for (int j = 1; j <= 100; j++) {
      x[3*j-2] = x[3*j-2] +
      y[3*j+2] = y[3*j-2] + 8
  }
  j = 301
}
```

INDUCTION VARIABLE SUBSTITUTION:

An induction variable is a variable commonly used in the context of loops. It is a variable that changes its value across loop iterations and is typically used to control the loop execution or to perform computations within the loop body:

```
for(int i = 1; i <= 100; i++){
    K = i (induction Var)
  for (int j = 1 : j <= 100; j++) {
      K = K + 2;
      x[3*j-2] = x[3*j-2] + K
      y[3*j+2] = y[3*j-2] + K
  }
  j = 301
}
```

→

```
for (int i = 1; i <= 100; i++) {
    K = i
  for (int j = 1; j <= 100; j++) {
      x[3*j-2] = x[3*j-2] +
                 K + 2*j
      y[3*j+2] = y[3*j-2] + K +
                 2*j
  }
  j = 301;
  K = K + 200;
}
```

EXPRESSION FOLDING AND FOWARD SUBSTITUTION

```
for (int i=1; i<=100; i++)
{
    k = i
  for (int j=1; j<=100; j++)
  {
     x[3*j-2] = x[3*j-2] + k+2*j ;
    y[3*j+2] = y[3*j-2] + k+2*j ;
  }
  j = 301 ;
  k = k + 200 ;
}
```

The k = i, can be removed and i can be directly substituted in the place of i.

} can be removed if they are not alive.

⇒ NORMALIZED CODE WITH LINEAR SUBSCRIPTS:

```
for (int i=1; i<=100; i++) {
    for (int j=1; j<=100; j++)
    {
       x[3*j-2] = x[3*j-2] + (i+2*j);
      y[3*j+2] = y[3*j-2] + (i+2*j);
    }
}
```

VECTOR CODE GENERATION:

Vector code generation, also known as vectorization, is a technique used to transform scalar code (which operates on individual data elements) into vectorized code.

For Eg:

```
for (int i = 1; i <= 100; i++)
{
    x[i] = x[i] + y[i];
}
```

Can be written in vector form as:

$$x(1:100) = x(1:100) + y(1:100)$$

Here x is an vector and y is also an vector.

⇒ Now only when there is no data dependence it is possible to vectorize the code.

Now for the previous example code:

$i = 1 \Rightarrow j = 1$, $S_1 : x[1] = x[1] + \cdots$

$S_2 : y[4] = y[1] + \cdots$

$j = 2$, $S_1 : x[2] = x[2] + \cdots$

$S_2 : y[7] = y[4] + \cdots$ RAW dependency.

Here S_1 can be vectorized but S_2 cannot be vectorized as there is Read after write [RAW] dependency.

```
for (int i=1 ; i<= 100; i++) {
    x[1:298:3] = x[1:298:3] + (i-2 : i+200 : 2);
 for (int j=1 ; j<= 100 ; j++) {
     y[3*j+2] = y[3*j-2] + i+2*j ;
 }
}
```

⇒ Partial vectorization.

DATA DEPENDENCE RELATIONS

There are totally 3 types of dependencies:

① WAW ② RAW ③ ~~R~~ WAR

① WAW (Write - after - write):

WAW dependence occurs when two or more instructions write to the same memory location or variable and the order of the write operations is important.

```
a = 5 (w)
b = 10
a = a+1 (w)
```

output dependence.

② RAW (Read - after - write):

RAW dependence occurs when one instruction reads a value produced by another instruction that writes to the same memory location or variable.

Eg: $a = 5$ $\quad X = \ldots . * (W)$

$b = a + 1$ $\quad \ldots = x \ (R)$ $\quad \uparrow$ (RAW)

Flow or true dependence.

③ WAR (Write after Read) : Anti-dependence.

$\ldots . = X \ (R)$

$X = \ldots . \ (W)$ $\uparrow$ WAR

DATA DEPENDENCE DIRECTION VECTOR

The data dependence direction vector, also known as DD Vector is a mathematical representation used to analyze and classify the dependencies between iterations of a loop. It provides information about the direction of data flow between different iterations in a loop.

-1: Indicates a Negative dependence. ($>$)
Data flows back ward in the loop iteration order (from a later iteration to an earlier iteration).

0: Indicates No dependence. ($=$)

1: Indicates a positive dependence: ($<$)
Data flows foward in the loop iteration.

Examples:

For negative dependence: (-1)

```
for (i=1 ; i< N; i++) {
    a[i-1] = a[i] + 1;
}
```

DD vector = [-1]

For No dependence (0):

```
for (i=1; i< N; i++) {
    a[i] = b[i] + c[i];
}
```

DD vector = [0]

For positive dependence (+):

```
for (i=1; i< N; i++) {
    a[i] = a[i-1] +1;
}
```

DD vector = [1]